Name : Sruthi Rathika .P

Reg no : 192521407

Course code : CSA1406

Course : compiler design

Date : 2.8.26

CO2 - AT 2

Q1. Parser validation :

Token stream : id + id * id

- Parser checks wheather the tokens follow the grammer rules.
- It gives * higher precedence than +.
- Expression is parsed as : id + (id * id)
- Hence, the expression is syntactically valid.

Q2. Derivation for aaabbb.

Grammar :

$S \rightarrow aSb \mid \varepsilon$

Derivation :

S

$\Rightarrow aSb$

$\Rightarrow aaSbb$

$\Rightarrow aaaSbbb$

$\Rightarrow aaabbb$

Therefore, aaabbb belongs to the language.

Q3. Remove Left Recursion:

Given:

$E \rightarrow E+T \mid T$

After removing left recursion:

$E \rightarrow T\,E'$

$E' \rightarrow +TE' \mid \varepsilon$

Q4. FIRST (A)

Grammar:

$A \rightarrow aB \mid \varepsilon$

$B \rightarrow b$

FIRST (A) = $\{a, \varepsilon\}$

Q5. Suitable for Top-Down parsing?

Grammar:

$A \rightarrow Aa \mid b$

No.

Reason:

- Contains left recursion.
- Recursive descent parser cannot parse it directly.

Q6. Left Factoring:

Given :

S → while E do S

| while E do S else S

After left factoring :

S → while E do SX

X → else S | ε

Q7. Parse Tree

Expression :

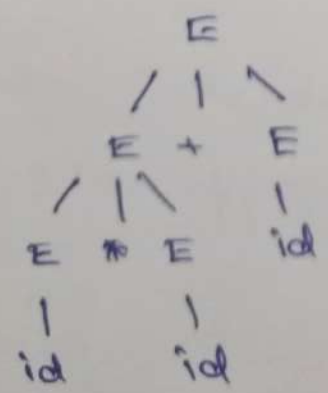
id * id + id

Grammar :

E → E + E

E → E * E

E → id



Q8. Recursive Descent procedures.

Grammar :

$S \rightarrow aA$

$A \rightarrow bA \mid c$

```
void S() {
    match ('a');
    A();
}
void A() {
    if (input == 'b') {
        match ('b');
        A();
    }
    else if (input == 'c')
        match ('c');
}
```

Q9. Predictive parsing Table.

Grammar :

$S \rightarrow AB$

$A \rightarrow a \mid \varepsilon$

$B \rightarrow b$

Non-terminal	a	b	c
S	$S \rightarrow AB$	$S \rightarrow AB$	
A	$A \rightarrow a$	$A \rightarrow \varepsilon$	

Q10. shift - Reduce parser (First Three Actions)

Input :

id + id

step	stack	Input	Action.
1		id + id $	shift id
2	id	+ id $	Reduce id → E
3	E	+ id $	shift +

Q11. Handle

Expression :

id * id + id

The first handle is :

id

(First reduced to E.)

Q12. Operator Precedence Relations.

Since * has higher precedence than + :

+ < *

* > +

+ = +

* = *

Q13. operator precedence parsing

Expression :

id + id * id

steps :

- Read id
- Rad +
- Read id
- Read *
- Since * has higher prededence, evaluate :

  id * id

- Then perform :

  id + ('id * 'id)

Q14. LR parser

- LR parser scans input. left - to - right.
- produces Rightmost derivation in reverse.
- uses shift and Reduce actions.

  id + id
  shift id
  Reduce id → E
  shift +
  Reduce id → E

Q15. Steps for constructing SLR Table

Grammer :

$S \rightarrow AA$

$A \rightarrow aA \mid b$

steps :

1. Augment grammer.
2. construct LR(0) items.
3. Find closure.
4. Compute GOTO.
5. Compute FOLLOW sets.
6. Fill ACTION Table.
7. Fill GOTO Table.

Q16. FOLLOW(A)

Grammer :

$S \rightarrow AB$

$A \rightarrow a \mid \varepsilon$

$B \rightarrow b$

Since B follows A,

FOLLOW(A) = {b}

Q17. Augmented Grammer

Original

$S \rightarrow aA$

$A \rightarrow b$

Augmented grammer :

$S' \rightarrow S$

$S \rightarrow aA$

$A \rightarrow b$

Q18. closure

Item

$S' \rightarrow . S$

Closure

$S' \rightarrow .S$

$S \rightarrow .aA$

Q19. Ambiguity

Grammar :

$E \rightarrow E + E \mid id$

string :

$id + id + id$

It has two parse trees :

$(id + id) + id$

id + (id+id)

Hence, the grammer is ambiguous.

Q20. Leftmost Derivation

Expression :

id + id * id

E
→E+E
→id+E
→id+E*E
→id+id*E
→id+id*id

Q21. Left factoring

Grammar :

S → aBa
| aBd
| b

After factoring

S → aBX | b
X → a | d

Q22. FIRST (F)

Grammar :

F → (E) | id

FIRST (F) = { (, id }

Q23. Suitable for Predictive parsing?

Grammar:

$S \rightarrow aA \mid aB$

$A \rightarrow b$

$B \rightarrow c$

No.

Reason:

- Both productions begin with a.
- FIRST sets overlap.
- Left factoring is required.

Q24. Recursive Descent Parser

Grammar:

$S \rightarrow abS \mid \varepsilon$

```
void S() {
    if (input == 'a') {
        match ('a');
        match ('b');
        S()
    }
}
```

Q25. Predictive parsing Table.

Grammar:

$S \rightarrow aS \mid b$

Non-terminal	a	b	$
S	S→aS	S→b	Error.